

MYSQL

Triggers e Stored Procedures



Exemplos Práticos: Banco de Dados Oficina

O que são Stored Procedures?

São blocos de código SQL salvos no banco de dados que podem ser reutilizados diversas vezes.



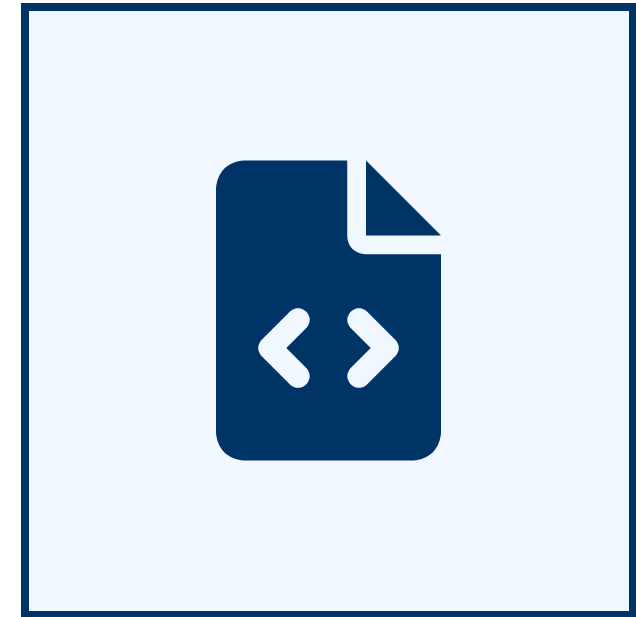
Redução de tráfego de rede: Menos comandos enviados ao servidor.



Melhor performance e segurança: Código pré-compilado e controle de acesso.



Lógica de negócios centralizada: Facilita a manutenção do sistema.



Criando uma Procedure (Exemplo Oficina)

Cenário: Registrar a entrada de um novo equipamento para serviço.

Vantagem: Garante que a data de entrada e o status 'Aberto' sejam padronizados.

```
DELIMITER //
```

```
CREATE PROCEDURE RegistrarEntrada(  
    IN p_cliente INT, IN p_equip INT, IN p_mecanico INT  
)  
BEGIN  
    INSERT INTO servico (data_entrada, status, id_mecanico, id_cliente,  
id_equipamento)  
    VALUES (CURDATE(), 'Aberto', p_mecanico, p_cliente, p_equip);  
END //
```

```
DELIMITER ;
```

Procedure: Cálculo de Valores (Oficina)

Cenário: Somar o preço de todos os reparos vinculados a um serviço específico.

Conceito: Uso de parâmetro de saída (OUT) para retornar o valor calculado.

```
CREATE PROCEDURE CalcularTotalServico(  
    IN p_id_servico INT, OUT p_total DECIMAL(10,2)  
)  
BEGIN  
    SELECT SUM(r.preco) INTO p_total  
    FROM reparo r  
    JOIN servico_reparo sr ON r.id_reparo = sr.reparo_id_reparo  
    WHERE sr.servico_id_servico = p_id_servico;  
END;
```

Como usar:

```
CALL CalcularTotalServico(10, @total_final);  
SELECT @total_final AS Valor_Total;
```

Executando Procedures



Para invocar uma procedure armazenada, utilizamos o comando **CALL** seguido do nome e parâmetros.

A execução processa a lógica interna no servidor, retornando apenas o resultado final.

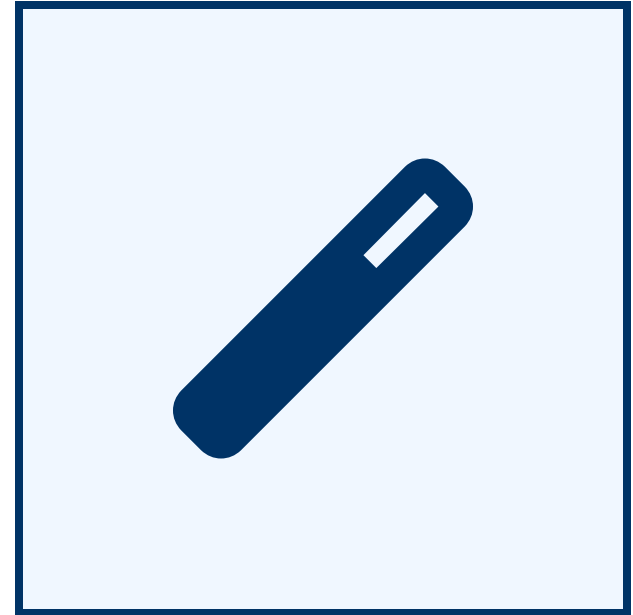
Exemplo de uso (Oficina):

```
-- Registrando serviço para:  
-- Cliente 1, Equipamento 2, Mecânico 3  
  
CALL RegistrarEntrada(1, 2, 3);
```

O que são Triggers?

Um **Trigger (Gatilho)** é um objeto do banco de dados disparado automaticamente em resposta a eventos.

- ⚡ Executados em eventos de **INSERT, UPDATE ou DELETE**.
- 🕒 Podem agir **BEFORE** (antes) ou **AFTER** (depois) do evento.
- 🛡️ Garantem **integridade de dados** e automação de **auditoria**.



Trigger: Validação de Negócio (Oficina)

Cenário: Impedir a exclusão de um cliente que possui serviços com status 'Aberto'.

Ação: Interrompe a operação de DELETE caso a condição seja verdadeira.

```
CREATE TRIGGER BeforeDeleteCliente
BEFORE DELETE ON cliente
FOR EACH ROW
BEGIN
    DECLARE v_count INT;
    SELECT COUNT(*) INTO v_count
    FROM servico
    WHERE id_cliente = OLD.id_cliente AND status = 'Aberto';

    IF v_count > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Erro: Cliente com serviços ativos.';
    END IF;
END;
```

Criando um Trigger (Exemplo Oficina)

Cenário: Atualizar automaticamente o estoque quando uma peça é usada em um reparo.

Ação: Após inserir em *reparo_peca*, subtrai 1 da tabela *peca*.

```
CREATE TRIGGER AfterInsertReparoPeca
AFTER INSERT ON reparo_peca
FOR EACH ROW
BEGIN
    UPDATE peca
    SET quantidade_estoque = quantidade_estoque - 1
    WHERE id_peca = NEW.peca_id_peca;
END;
```

Procedures vs Triggers

Característica	Stored Procedure	Trigger
Execução	Chamada manual (CALL)	Automática (Evento)
Parâmetros	Aceita IN, OUT, INOUT	Não aceita parâmetros
Retorno	Pode retornar resultados	Não retorna resultados
Uso Comum	Lógica de Negócio	Auditoria e Integridade

Boas Práticas e Conclusão

- ✓ Use **Procedures** para lógica de negócios complexa e operações que exigem múltiplos passos SQL.
- 🛡 Use **Triggers** para garantir integridade referencial e auditoria, mas evite lógicas excessivamente complexas.
- 📖 Sempre **documente** a lógica interna e os parâmetros para facilitar a manutenção futura.
- 📊 Monitore a **performance**: gatilhos em tabelas de alta transação podem causar lentidão no banco.

**Automação inteligente
gera sistemas robustos e
confiáveis.**



Fim da Apresentação - Oficina Mecânica SQL